

Drone AI – Face Detection (DJI Tello Experiments)

Project: Drone AI - Face Detection

Date: 10/4/2026

Team: Team 1, Monday

Khoa Do	khoado@csu.fullerton.edu
John Alora	jtalora@csu.fullerton.edu
Samson Du	samsondu888@csu.fullerton.edu
Wen Fan	wenfan@csu.fullerton.edu
Arian Gholamipour	arian-gh@csu.fullerton.edu
Oscar De La Torre	oscar_delatorrej@csu.fullerton.edu

Discussion

This project combines three practical components of edge AI: a DJI Tello drone, real-time video streaming, and a prebuilt computer-vision face detector. Its purpose is not to identify *who* a person is. Instead, it performs face **detection**: for each video frame, it determines whether one or more faces are present and, when possible, returns their locations as bounding boxes. This distinction matters. Detection answers “is there a face and where is it?” whereas recognition would require an enrolled database and would answer “whose face is this?” The use of a programmable drone with a live camera stream provides a practical way to connect computer vision with physical movement. Previous work has also demonstrated the Tello as a platform for real-time computer-vision applications involving facial detection and autonomous control.

The project begins with the drone-control program in `DroneConnnect.py`. The program creates a `Tello` object from the `djitellopy` library, connects to the Tello Wi-Fi access point, and prints the battery level. It then takes off, moves upward by 80 cm, and sends an RC-control command with a positive yaw value for five seconds. Thus, the drone rotates in place rather than translating horizontally. Sending `(0, 0, 0, 0)` stops the rotation, and `land()` completes the flight. This small control

experiment is valuable because it verifies connectivity, battery communication, takeoff and landing, distance movement, and velocity-based control before visual AI is added. Testing these functions separately reduces the chance that a flight problem will be confused with a face-detection problem.

The use of `djitellopy` provides a Python interface between the computer program and the Tello drone. This interface allows the program to establish a connection with the drone, retrieve frames from its camera stream, and send flight commands. The library is useful for this project because it allows the camera stream and drone-control functions to be accessed through Python. Priambodo et al. describe a similar use of the DJI Tello in which `djitellopy` was used to communicate with the drone, receive its live video stream, and send movement commands based on computer-vision process. Their system used Python and OpenCV to process the camera stream, with the information from the detected face being used to calculate the drone movements. This demonstrates how Tello's video and flight-control capabilities can be integrated into a computer-vision application. In the current project, `djitellopy` provides the connection between the face-detection system and the physical drone, allowing the detection result to trigger a programmed flight action. (Priambodo et al., 2021)

The face-detection programs, `FaceDetectSimple.py` and `FaceDetectionExample.py`, build the perception part of the system. After connecting to the drone, the code calls `streamoff()` and `streamon()` to reset and start the Tello video stream. Within the continuous loop, `me.get_frame_read().frame` retrieves the most recent camera image. That image is supplied to cvzone's `FaceDetector` through `detector.findFaces(img, draw=True)`. cvzone is useful here because it packages MediaPipe-based vision functionality behind a simple interface. Rather than implementing a neural network and post-processing algorithm from scratch, the group can focus on system integration: receiving a live frame, running inference, inspecting the result, and deciding how the drone should react.

The detector returns two values. The first is an image, optionally annotated with detection graphics; the second, `bboxs`, is a list of face bounding-box records. The key design feature is that `bboxs` can be empty or can contain multiple faces. Therefore, `if bboxs:` is the correct condition for deciding that a face has been detected. The camera-only example shows more explicitly what a record can provide: a bounding box (`x, y, width, height`), a center point, and a confidence score. It uses `cvzone.cornerRect`, `cvzone.putTextRect`, and `cv2.circle` to show the box, percentage score, and center point. These visual overlays are useful for debugging

because the group can see whether the detector is responding to an actual face and whether the box follows that face correctly.

The bounding-box information is an important part of computer-vision output because it provides more information than simply returning a true or false detection. The coordinates indicate where the face is located within the camera frame, while the width and height provides information about the apparent size of the detected face. The center point can also be used to determine the face's position relative to the center of the camera image. Priambodo et al. demonstrate how the position of a detected face can be compared with a set point in the center of the camera image to calculate positional error for drone movement (Priambodo et al., 2021). In the current project, these additional values are displayed and can be used to observe the detector's behavior, but the flight decision only checks whether `bboxes` contain at least one detected face. These values could support future development, such as controlling yaw based on the horizontal position of a face or adjusting movement based on its apparent size.

An important consideration when using face detection to control a physical system is the possibility of false positives and unstable detections. Computer-vision detectors may occasionally detect or identify an object incorrectly and produce bounding boxes that change between frames. Ryan discusses false positives and detection jitter as a challenge when using facial detection for Tello flight control (Ryan, 2020). This issue is relevant to the current project because a false detection could cause the drone to perform the jump even when a real face is not present.

OpenCV provides the display and user-control layer. `cv2.imshow()` presents each annotated frame in a window, while `cv2.waitKey(5)` briefly processes keyboard events and refreshes the display. Pressing `q` breaks the loop, after which `cv2.destroyAllWindows()` closes the OpenCV windows. This loop is an edge-AI pattern: camera acquisition, inference, visualization, and a control decision repeat continuously on the laptop connected to the drone. The resulting system is responsive because it does not need to upload the video stream to a remote server before acting.

The completed `faceDetectJump.py` implements the required Step 4 perception-to-action rule. After takeoff, the program monitors `bboxes`; when at least one face is detected, the drone flies higher for three seconds and then returns to its original elevation. The core decision is therefore `if bboxes:`, followed by an upward velocity command, a three-second delay, a stop command, an equal-duration downward command, and another stop. The important engineering detail is that the response should be treated as an event rather than as a command that is repeated for every

processed frame. Because the face detector operates continuously, the same face could appear in many consecutive frames. Without state management, the drone could attempt to perform jumps repeatedly. A Boolean state variable can mark that the jump has already occurred, and a short cooldown or a requirement that the face disappear before re-arming prevents repeated motion while the same person remains in view. This transforms a simple face detector into a small autonomous reactive system.

The architecture of the project can also be understood as several connected layers. The Tello provides a physical platform and camera input. `djitellopy` provides communication between Python and the drone, allowing the program to receive video frames and send flight commands. The face detector processes each camera image and produces a face-detection result containing all the information, such as bounding boxes. OpenCV provides visualization and keyboard interaction, allowing the detected faces to be displayed and the program to be stopped using the q key. Finally, the flight-control code converts the detection result into an action. When the `bboxes` is not empty, the drone performs the programmed vertical movement. Separating these components makes the system easier to test and troubleshoot because the problems can be isolated to the drone connection, video stream, face detection, visualization, or flight-control logic.

Testing the completed program in the classroom is an essential part of the project rather than merely a final demonstration. The group should first confirm that the Tello connection and video stream are stable, then confirm that a face is visibly outlined in the OpenCV window, and only then test the flight response. During the test, the group can observe whether the detection occurs consistently at normal viewing distance, whether the drone rises for approximately the intended duration, and whether it returns to a safe height. Repeating the test under different lighting conditions and with the face entering and leaving the camera frame will help distinguish a reliable reaction from a one-time success. This practical validation matters because visual detection is affected by camera angle, distance, blur, shadows, and wireless-stream latency.

The completed solution also shows the relationship between high-level AI output and low-level flight control. `FaceDetector` supplies a semantic result—a face is present—while `send_rc_control()` accepts four numeric control values for left/right, forward/backward, up/down, and yaw movement. The program is the bridge between them. In this assignment, the mapping is deliberately simple: the presence of any face produces a vertical “jump.” More advanced versions could use the bounding-box center to rotate toward a face, box size to estimate whether the person is too close or too far away, or multiple consecutive detections to reduce false triggers. Those extensions are outside the required scope, but the current design establishes the same

perception–decision–action architecture used in more sophisticated autonomous systems.

Safety is central when connecting AI output to a physical drone. The code should verify battery level before flight, test in the instructor’s controlled classroom area, keep people clear of propellers, and use small, conservative movements. Most importantly, landing should occur in a **finally** block or another cleanup path so that an exception or early exit does not leave the drone airborne. Ryan also discusses the importance of safely terminating a Tello application and sending a landing command when the program exits (Ryan, 2020). So a safe cleanup and shutdown behavior should be considered when designing the system.

Detection confidence and false positives should also be considered when evaluating the completed system. A face-like image may trigger a detection, while poor lighting, motion blur, or partial occlusion may prevent a face from being detected. Priambodo et al. similarly note that faces may not be detected under poor lighting or when required facial features are not sufficiently visible (Priambodo et al., 2021). These limitations are important in this project because the presence of a detected face can trigger the drone’s vertical movement. An incorrect detection can result in an unintended flight response. This experiment is therefore a strong first prototype of an autonomous drone behavior, while also showing why reliable state handling, classroom testing, and safety constraints are required before more advanced tracking or navigation is attempted.

Our Closing Thoughts

Overall, this experiment demonstrated how a relatively simple computer-vision model can be integrated with a programmable drone to perform an autonomous task. The Tello provides the physical platform and camera stream, **djitellopy** provides the communication interface, cvzone provides a convenient face-detection interface, and OpenCV provides visualization and user interaction. This project also demonstrates important engineering considerations, including the use of bounding-box information, the possibility of false positive and detection instability, and the need for reliable program termination. Overall, the project demonstrates how Edge AI techniques can turn real-time visual information into physical actions and provide hands-on experience with computer vision, programming, and drone control.

References

- Priambodo, A. S., Arifin, F., Nashua, A., & Winursito, A. (2021). *Face Tracking for Flying Robot Quadcopter based on Haar Cascade Classifier and PID Controller*.

Journal of Physics: Conference Series, 2111, 012046.

<https://iopscience.iop.org/article/10.1088/1742-6596/2111/1/012046/meta>

- Ryan P. (2020, June 14). *Tello Drone Face Follower*. The Startup.
<https://medium.com/swlh/tello-drone-face-follower-4b8313b6c40e>
- Project source files: `DroneConntect.py`, `FaceDetectSimple.py`, `FaceDetectionExample.py`, `CornerRectangleExample.py`, and `faceDetectJump.py`.
- [cvzone GitHub repository](#)